

VOLUME 08 / RUST BACKEND MASTERY

認証・認可 — Topcoat Session・Argon2id・Better Auth 比較

守るもの・攻める人・入口を先に書き出し、パスワード・セッション・CSRF・他人のデータの守りを、手を動かして確かめる

第8巻: 認証・認可 — Topcoat Session・Argon2id・Better Auth 比較

この巻で身に付けること

ログインの仕組みは、「メールアドレスとパスワードを入れたら、中に入れる」だけに見えます。ところが、その裏では、たくさんのことを守っています。守り方を1つまちがえると、次のような事故が起きます。

- DB のファイルが盗まれ、全員のパスワードがそのまま読まれてしまう
- ログインの画面の文の違いから、「このメールアドレスは登録されている」と他人に分かってしまう
- ほかのサイトに置かれたボタンを押しただけで、自分のノートが消えてしまう
- URL の数字を1つ変えただけで、他人のノートが見えてしまう
- ログアウトしたのに、前のクッキーを使えば、まだ中に入れてしまう

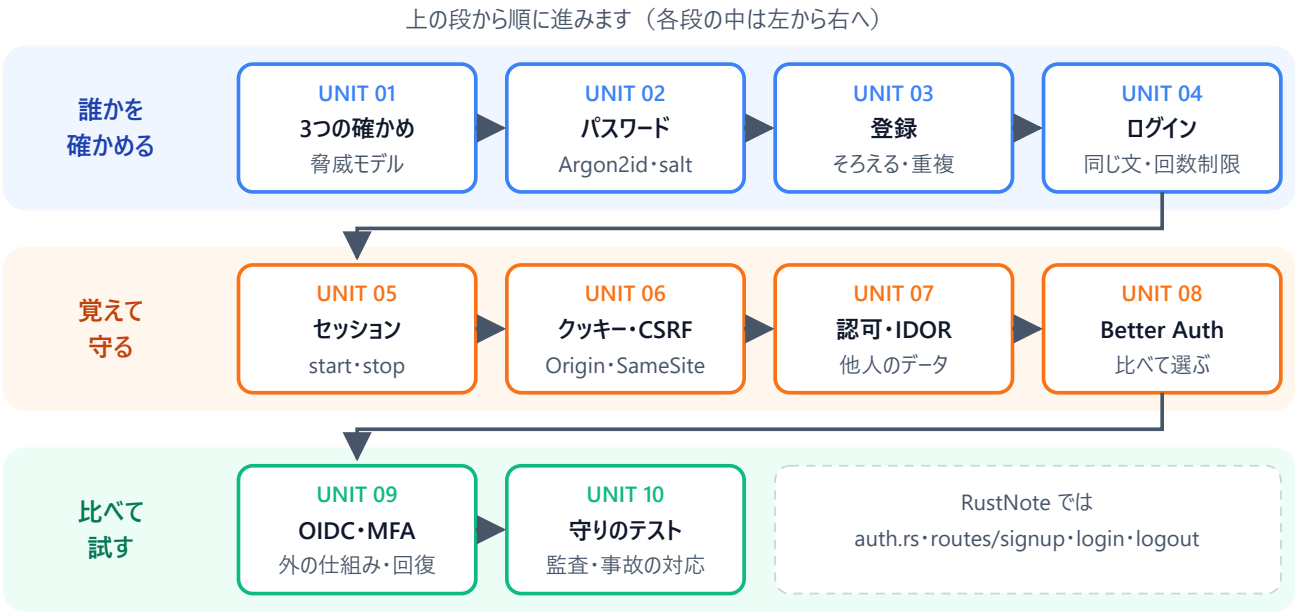
この巻では、こうした事故を、**先に「何を守るのか・誰が攻めるのか・どこから入られるのか」を書き出す**ことで防ぎます。そのうえで、RustNote が実際にどう守っているかを、テストを動かして1つずつ確かめます。

到達目標

この巻を終えると、次のことができるようになります。

- 認証（あなたは誰か）・セッション（さっきの人と同じか）・認可（それをしてよいか）を区別し、それぞれを RustNote のどこで確かめているかを言える
- 守るもの・攻める人・入口・信頼の境目を書き出した「脅威モデル」を作り、それぞれの攻撃に対する守りを表にできる
- パスワードを Argon2id でハッシュにして保存し、登録とログインで、メールアドレスの存在が他人に分からないようにできる
- Topcoat のセッション（`session::start`・`stop`）と、クッキーの属性（`HttpOnly`・`Secure`・`SameSite`）、CSRF の守り（Origin の検査）を説明し、テストで確かめられる
- 他人のデータを見せない（IDOR を防ぐ）書き方と、Better Auth のような外のサービスを使う場合との違いを説明できる

学習地図



各単元の流れ: たとえ話 → 図 → 最小コード → 実務の形 → よくあるエラー → 練習問題

図8-1: 第8巻の学習地図。誰なのかを確かめ、それを覚えて守り、ほかのやり方と比べて、守りをテストで確かめる

単元	テーマ	ひとことで言うと
UNIT 01	認証・セッション・認可と脅威モデル	3つの確かめを区別し、守るもの・攻める人・入口を書き出す
UNIT 02	パスワード・Argon2id・salt	パスワードは「戻せない形」にして保存する
UNIT 03	登録・メールのそろえ方・重複（準備中）	同じ人が2つ登録できないようにする
UNIT 04	ログイン・同じ文で断る・回数の制限（準備中）	失敗の理由を、他人に教えない
UNIT 05	Topcoat のセッションの一生（準備中）	発行・照合・ログアウトを、DB の記録と組にする
UNIT 06	クッキー・CSRF・Origin・SameSite（準備中）	ほかのサイトからの送信を止める
UNIT 07	認可・IDOR・持ち主の境目（準備中）	URL の ID を変えても、他人のデータは見せない
UNIT 08	Better Auth との比較と、おすすめの構成（準備中）	自分で作るか、外の仕組みを使うかを決める
UNIT 09	OIDC・外の認証サービス・MFA・回復（準備中）	ソーシャルログインや二段階認証を考える
UNIT 10	守りのテスト・監査・事故の対応（準備中）	攻撃のテストを自動で回し、記録を残す

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03〜10 は準備中で、次の更新から順に追加します。

前提知識（先に読んでおく巻）

- **第4巻（設計）**：newtype（`UserId`・`NoteId`）と、不正な値を作れない型（第4巻 UNIT 01）。UNIT 01 のエラー2で、型が「他人のデータ」の事故を防ぐ様子を見ます。
- **第7巻（Turso・libSQL）**：持ち主の条件を付けた SQL（第7巻 UNIT 03）。
- **第9巻（RustNote 設計）**：受入条件のテスト（`tests/` に足して `cargo test --test 名前` で動かす形）と、画面の遷移（PRG・303）。この巻のテストも、同じ形で書きます。

> **注意:** 第6巻（Topcoat）は、書き直しの途中です。この巻で使う Topcoat の機能（セッション・クッキー・Origin の検査）は、本文の中で、そのつど説明します。

この巻の進め方

守りの仕組みは、「うまくいく場合」を見ても、ちゃんと働いているかどうかが分かりません。各単元では、次の順番で進めてください。

1. **攻め方を予想する:** 「どうすれば、他人のデータを見られるか」を、攻める人の気持ちで考えます。
2. **攻めてみる:** 教材のテストで、実際にその攻撃を送ってみます（自分の PC の中だけで行います）。
3. **止まったことを確かめる:** ステータスコード（303・401・403・404）と、DB に何も残っていないことを確かめます。
4. **守りを1つ外したらどうなるか:** 「よくあるエラーと読み方」の節で、守りが無いときの様子を見ます。

> **注意:** この巻の攻撃の実験は、すべて自分の PC の中の、練習用の RustNote に対して行います。他人のサイトやサービスに対して試すことは、法律で禁じられている場合があります。絶対にしないでください。

使うツールと版

この巻のコードは、すべて次の環境で実際にコンパイル・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.98 以上**（本書シリーズの基準）。動作確認は rustc 1.99.0 / cargo 1.99.0（`aarch64-pc-windows-msvc`）
RustNote	教材の `Rust/rustnote/`（`rustnote` 0.1.0）。この巻では `src/` を変えずに、`tests/` と `examples/` に確かめ用のファイルを足す
topcoat	0.10.0（セッション・クッキー・Origin の検査）
argon2	0.6.0（パスワードのハッシュ。UNIT 02 でくわしく扱います）

練習用のフォルダを用意する

第9巻と同じように、教材の `Rust/rustnote/` を**作業用のフォルダにコピー**して使います。

```
# 教材の rustnote フォルダを、中身ごと（-Recurse）作業用のフォルダへコピーする
Copy-Item -Recurse "教材のフォルダ\Rust\rustnote" .\rustnote
cd rustnote
# このフォルダで使われる Rust の版を確かめる（1.99.0 になる）
rustup show active-toolchain
```

この巻のテストは、RustNote の `tests/common/mod.rs` にある道具（`test_app`・`sign_up`・`create_note` など）を使います。テストごとに一時フォルダに新しい DB を作り、ポートを開かずに、ルーターへ直接リクエストを渡して動かしま

す（第9巻 UNIT 01）。

UNIT 01: 認証・セッション・認可と脅威モデル

この単元のゴール

- 認証（あなたは誰か）・セッション（さっきの人と同じか）・認可（それをしてよいか）の3つの確かめを区別し、RustNote でそれぞれが失敗したときの返事（303・404）を、テストで確かめられる
- ブラウザに渡すクッキー（トークン）と、DB に保存するもの（トークンのハッシュ）が違うことを、実際の値で説明できる
- ほかのサイトから送られてきたフォーム（CSRF）を、Topcoat が 403 で止めることを確かめられる
- 守るもの・攻める人・入口・信頼の境目を書き出した「脅威モデル」を作り、RustNote の守りと、まだ無い守りを表にできる

たとえ話でつかむ

会員制のコンサート会場を考えます。

- **入口の受付**で、会員証と顔写真を見て「この人は本人だ」と確かめます。これが**認証**です。
- 本人だと分かったら、**リストバンド**を腕に付けてもらいます。会場の中では、毎回会員証を見せなくても、リストバンドを見せれば「さっき受付を通った人だ」と分かります。これが**セッション**です。
- 楽屋の前の係は、リストバンドを見て「会場に入ってよい人だ」と分かっても、それだけでは通しません。「この人は、この楽屋に入ってよいか」を、別に確かめます。これが**認可**です。

会場の警備の担当者は、開場の前に**警備の計画**を立てます。「守るもの（お客さんの財布・楽屋の機材）」「攻めてくる人（偽の会員証を作る人・落ちたリストバンドを拾った人・外から呼び出して入口を開けさせる人）」「入口（正面・裏口・搬入口）」を書き出してから、どこに係を立てるかを決めます。この計画が**脅威モデル**です。

コンサート会場	RustNote
受付で会員証を確かめる	**認証**（メールアドレスとパスワードで、本人かを確かめる）
リストバンド	**セッション**（クッキーの `__Host-session` に入ったトークン）
リストバンドの台帳（番号の控え）	DB の `sessions` 表（トークンそのものではなく、そのハッシュを控える）
楽屋の前の係	**認可**（このノートの持ち主か。SQL の `user_id = ?`）
警備の計画	**脅威モデル**（守るもの・攻める人・入口・信頼の境目）
外から呼び出して入口を開けさせる	**CSRF**（ほかのサイトから、ログイン中の人のブラウザにフォームを送らせる）

> **用語:** 認証（にんしょう。authentication：相手が「名乗っているとおりの人か」を確かめること）

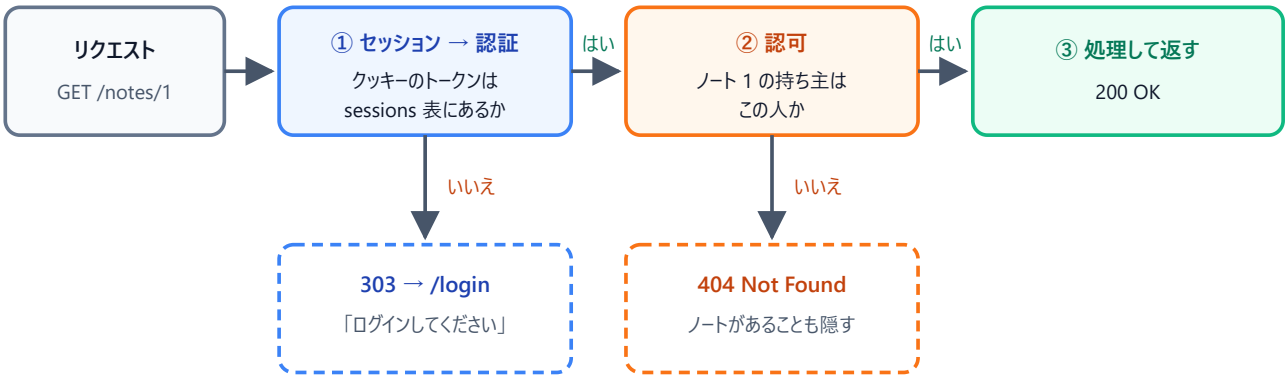
> **用語:** 認可（にんか。authorization：確かめた相手に、「その操作をしてよいか」を決めること）

> **用語:** セッション（session：一度認証した人を、次のリクエストからも「同じ人」と分かるようにする仕組み。RustNote ではクッキーのトークンで行う）

> **用語:** 脅威モデル（きょういモデル。threat model：守るもの・攻める人・入口・信頼の境目を書き出し、どんな攻撃がありうるかを整理したもの）

図で理解する

RustNote にリクエストが届くと、次の順番で確かめます。



① は auth.rs の require_user、② は repository の SQL の user_id = ? が受け持つ

図8-2: RustNote の確かめの順番。セッションから「誰か」が分からなければ 303 でログイン画面へ、持ち主でなければ 404 にする

- **① セッションと認証:** ブラウザが送ってきたクッキーのトークンから、ログイン中のユーザーを探します。見つからなければ、「ログインしてください」を出すために、303 でログイン画面へ移します（`auth.rs` の `require_user`）。
- **② 認可:** ユーザーが分かっても、そのノートの持ち主かどうかは別に確かめます。RustNote は、ノートを探す SQL に必ず `user_id = ?`（持ち主の条件）を入れるので、他人のノートは「見つからない」ことになり、404 を返します。「ある」ことを知らせる 403 にしないのは、他人のノートの番号を探り当てられないようにするためです（第9巻 UNIT 01）。

もう1つの図は、RustNote の脅威モデルです。誰が、どこから、何を狙うのかを描きます。

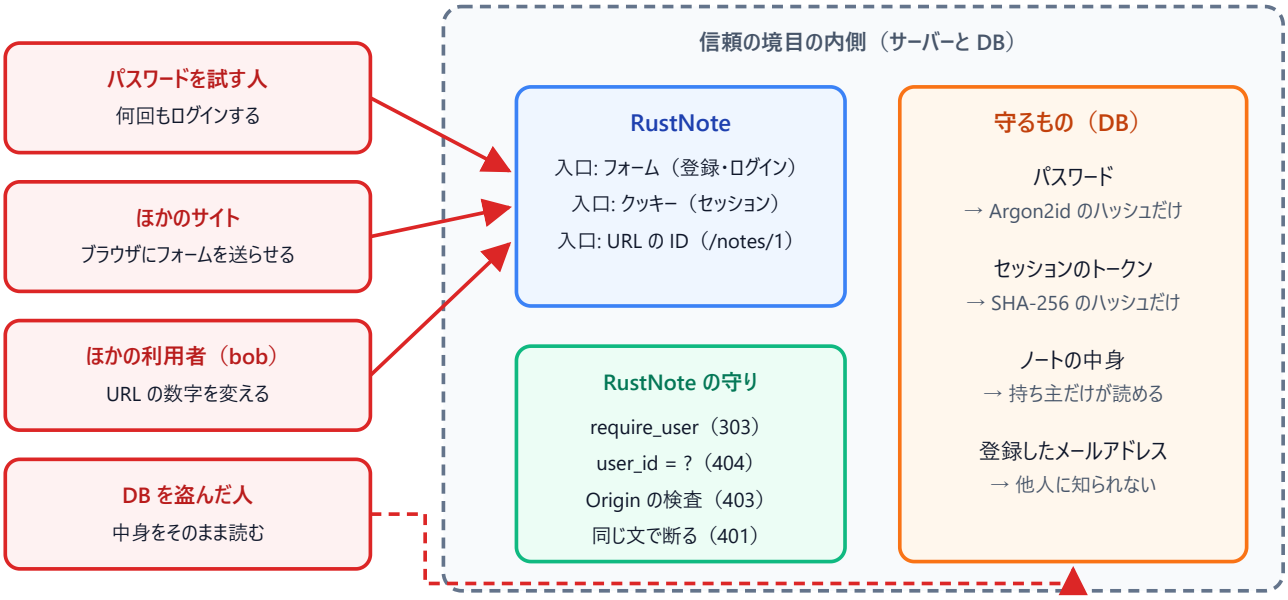


図8-3: RustNote の脅威モデル。左の4種類の「攻める人」が、3つの入口（フォーム・クッキー・URL の ID）から、DB の中の守るものを狙う。DB を盗んだ人は入口を通らないので、保存する形そのもので守る

- **入口は3つです。**フォーム（登録・ログイン・ノートの作成）、クッキー（セッションのトークン）、URL の ID（`/notes/1` の `1`）。攻める人は、この3つのどれかに、変な値を入れてきます。
- **DB を盗んだ人**だけは、入口を通りません。だから、パスワードとトークンは、盗まれても元に戻せない形（ハッシュ）で保存します。「入口で止める」守りと、「盗まれても困らない形で持つ」守りの2種類が必要なのです。
- 点線の四角が**信頼の境目**です。境目の外（ブラウザ・ほかのサイト）から来た値は、どれも信じずに、内側で確かめ直します。

まず動かす最小コード

例1-1: 3つの確かめを、1つのテストで見る

> **▼ このコードがやること（先に日本語で）**: ログインしていない人が `/notes` を開く（①認証が無い）、alice が登録してクッキーを付けて開く（②セッションで本人と分かる）、alice が作ったノートを alice と bob がそれぞれ開く（③認可）の3つを、順に試します。それぞれの返事を表示します。

```rust
//! 第8巻 UNIT 01: 認証・セッション・認可の3つの確かめを、RustNote で1つずつ見る。
//! 実行: cargo test --test u01_three_checks -- --nocapture --test-threads=1

mod common;

use common::{location, test_app};
use topcoat::router::StatusCode;

const PASSWORD: &str = "correct horse battery";

#[tokio::test]
async fn three_checks() {
    let app = test_app().await;

    // ① 認証: 誰なのか分からない（ログインしていない）人
    let response = app.get("/notes", None).await;
    println!(
        "① ログインしていない人が /notes → {}（移動先 {}）",
        response.status(),
        location(&response)
    );

    // ② 認証とセッション: 登録した alice は、クッキーを見せれば「alice だ」と分かる
    let alice = app.sign_up("alice@example.com", PASSWORD).await;
    let response = app.get("/notes", Some(&alice)).await;
    println!("② alice がクッキーを付けて /notes → {}", response.status());

    // ③ 認可: bob はログインしているが、alice のノートを開いてよいか
    let url = app.create_note(&alice, "alice の買い物メモ", "牛乳").await;
    let bob = app.sign_up("bob@example.com", PASSWORD).await;
    let by_owner = app.get(&url, Some(&alice)).await.status();
    let by_other = app.get(&url, Some(&bob)).await.status();
    println!("③ {url} を alice が開く → {by_owner}");
    println!("③ {url} を bob が開く → {by_other}");

    assert_eq!(by_owner, StatusCode::OK);
    assert_eq!(by_other, StatusCode::NOT_FOUND);
}
```

コピーした `rustnote` フォルダの `tests/u01_three_checks.rs` として保存して実行します。`--nocapture` で `println!` を表示し、`--test-threads=1` で、表示が混ざらないようにします。

```
cargo test --test u01_three_checks -- --nocapture --test-threads=1
```

▼ **実行結果**（最初の1回は `Compiling` の行が続きます。ここでは省いています）

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 1m 15s
     Running tests\u01_three_checks.rs (target\debug\deps\u01_three_checks-
66387e9ac1880c66.exe)

running 1 test
test three_checks ... GET /notes -> 303 See Other (1.3467ms)
① ログインしていない人が /notes → 303 See Other（移動先 /login）
POST /signup -> 303 See Other (438.8671ms)
GET /notes -> 200 OK (4.1174ms)
② alice がクッキーを付けて /notes → 200 OK
POST /notes -> 303 See Other (4.9204ms)
POST /signup -> 303 See Other (454.0788ms)
GET /notes/1 -> 200 OK (4.833ms)
GET /notes/1 -> 404 Not Found (3.6291ms)
③ /notes/1 を alice が開く → 200 OK
③ /notes/1 を bob が開く → 404 Not Found
ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.94s
```

▼ コードを1つずつ分解して解説

解説1: ① 認証が無いと、303 でログイン画面へ

- クッキーを付けずに `/notes` を開くと、`303 See Other`（移動先 `/login`）でした。RustNote のノートの画面は、どれも最初に `require_user(cx)` を呼び、ログイン中のユーザーがいなければ、ログイン画面へ移します。
- 1行目の `test three_checks ... GET /notes -> 303 See Other (1.3467ms)` は、テストの名前の表示と、RustNote のアクセスログ（標準エラー出力）が、同じ行に並んだものです。

解説2: ② クッキーを見せると、本人と分かる

- `app.sign_up(...)` は、登録のフォームを送り、返事の `Set-Cookie` から `__Host-session=...` を取り出して返す、テスト用の道具です（`tests/common/mod.rs`）。そのクッキーを付けて `/notes` を開くと、`200 OK` でした。
- 2回目からは、パスワードを送っていません。クッキーのトークンだけで「alice だ」と分かったのです。これがセッションの役目です。

解説3: ③ ログインしていても、他人のノートは 404

- alice のノート（`/notes/1`）を alice が開くと `200 OK`、bob が開くと `404 Not Found` でした。bob はログインしている（認証は通っている）のに、認可で断られたのです。
- 認証と認可は、別の確かめです。「ログインしているから、何でも見てよい」わけではありません。

もう一歩: 実務に近い形

例1-2: クッキーに入るものと、DB に保存されるものを比べる

> **▼ このコードがやること（先に日本語で）**: 登録のフォームを送り、返事の `Set-Cookie` から、セッションのクッキーの名前・値（トークン）の先頭・属性を表示します。続いて、DB の `sessions` 表に入った行（トークンのハッシュの長さ・持ち主・期限）を表示し、トークンそのものと同じ値が DB に入っていないことを確かめます。

```rust
//! 第8巻 UNIT 01: セッションのクッキーに入るものと、DB に保存されるものを比べる。
//! 実行: cargo test --test u01_session -- --nocapture

mod common;

use common::test_app;
use topcoat::router::header;

#[tokio::test]
async fn session_cookie_and_record() {
    let app = test_app().await;
    let form =
"email=alice%40example.com&password=correct+horse+battery&password_confirm=correct+hors
e+battery";
    let response = app.post_form("/signup", form, None).await;

    // Set-Cookie の中から、セッションのクッキーを探す
    let set_cookie = response
        .headers()
        .get_all(header::SET_COOKIE)
        .iter()
        .filter_map(|value| value.to_str().ok())
        .find(|value| value.starts_with("__Host-session="))
        .expect("セッションのクッキーがありません")
        .to_string();
    let (pair, attributes) = set_cookie.split_once(';').expect("属性がありません");
    let token = pair.trim_start_matches("__Host-session=");
    println!("--- ブラウザに渡すクッキー");
    println!("名前: __Host-session");
    println!("値（トークン）: {}…（{} 文字）", &token[..6], token.len());
    println!("属性:{attributes}");

    // DB の sessions 表に入っているもの
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    let mut rows = conn
        .query(
            "SELECT length(token_hash), user_id, expires_at - unixepoch() FROM
sessions",
            (),
        )
        .await
        .expect("sessions を読めません");
    println!("--- DB の sessions 表");
    while let Some(row) = rows.next().await.expect("行を読めません") {
        let bytes: i64 = row.get(0).expect("token_hash の長さ");
        let user_id: i64 = row.get(1).expect("user_id");
        let seconds: i64 = row.get(2).expect("期限までの秒数");
        println!(
            "token_hash: {bytes} バイト / user_id: {user_id} / 期限まで: 約 {} 日",
```

```
            seconds / 86_400
        );
    }

    // トークンそのものは、DB のどこにも入っていない
    let mut rows = conn
        .query(
            "SELECT count(*) FROM sessions WHERE token_hash = CAST(?1 AS BLOB)",
            [token],
        )
        .await
        .expect("sessions を読めません");
    let row = rows
        .next()
        .await
        .expect("行を読めません")
        .expect("行がありません");
    let same: i64 = row.get(0).expect("件数");
    println!("トークンと同じ値が入っている行: {same} 件");
}
```

```
cargo test --test u01_session -- --nocapture
```

▼ 実行結果

```
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.31s
     Running tests\u01_session.rs (target\debug\deps\u01_session-7d82e6b13d3770c3.exe)

running 1 test
POST /signup -> 303 See Other (421.6028ms)
--- ブラウザに渡すクッキー
名前: __Host-session
値（トークン）: qPxZTv…（46 文字）
属性: HttpOnly; SameSite=Lax; Secure; Path=/; Max-Age=2592000
--- DB の sessions 表
token_hash: 32 バイト / user_id: 1 / 期限まで: 約 30 日
トークンと同じ値が入っている行: 0 件
test session_cookie_and_record ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.45s
```

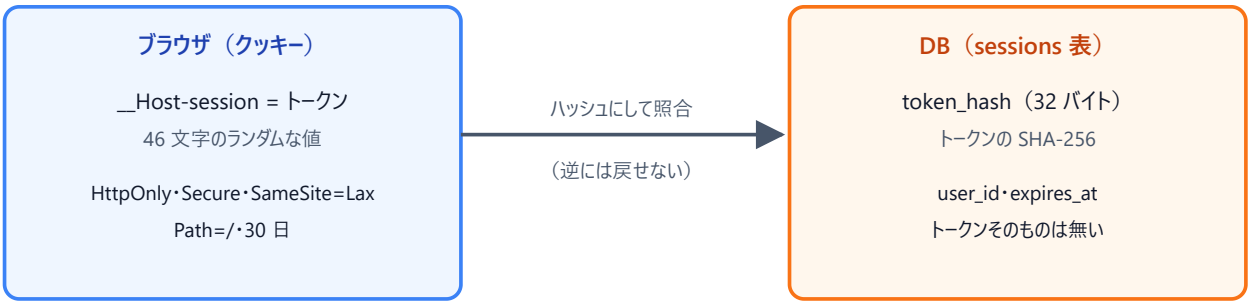


図8-4: クッキーに入るトークンと、DB に保存されるハッシュ。DB にはトークンそのものが無いので、DB を盗まれても、そのままではログイン中の人になりすませない

▼ コードを1つずつ分解して解説

解説1: クッキーの名前と属性が、トークンを守っている

- `__Host-session` の `__Host-` は、ブラウザに「`Secure` が付いていて、`Path=/` で、ほかのドメインと共有しないクッキーだけを受け付けよ」と求める、特別な名前の付け方です。名前の決まりを守らないクッキーは、ブラウザが受け付けません。
- `HttpOnly` は「ページの JavaScript からは読めない」、`Secure` は「HTTPS のときだけ送る」、`SameSite=Lax` は「ほかのサイトからの POST には付けて送らない」という意味です。`Max-Age=2592000` は 30 日（2,592,000 秒）で消えるという意味です。属性の1つずつは、UNIT 06（準備中）でくわしく扱います。
- これらの属性は、Topcoat の `SessionConfig::default()` が付けています（RustNote の `routes.rs` の `.sessions(SessionConfig::default())`）。

解説2: DB には、トークンではなくハッシュを保存する

- `sessions` 表の `token_hash` は 32 バイトでした。Topcoat が、トークンから SHA-256（1方向にしか計算できない変換）で作った値です。クッキーの値（46 文字）とは、長さも中身も違います。最後の行で、トークンと同じ値の行が 0 件であることも確かめました。
- RustNote の `auth.rs` の `log_in` は、`session::start(cx)` でトークンを発行してクッキーに入れ、返ってきた `session.token_hash` だけを `sessions` 表に保存します。次のリクエストでは、クッキーのトークンを同じようにハッシュにして、表の中を探します（`load_current_user`）。
- こうしておくと、DB のファイルが盗まれても、表に入っているのはハッシュだけなので、それをクッキーに入れてもログインできません（問3）。

例1-3: ほかのサイトから送られてきたフォームを、Topcoat が止めるか

ログイン中の人のブラウザには、RustNote のクッキーが入っています。悪いサイトが、そのブラウザに「RustNote の `/notes` へ POST するフォーム」を自動で送らせたら、どうなるでしょう。これが **CSRF**（クロスサイト・リクエスト・フォージェリ）という攻撃です。

▼ **このコードがやること（先に日本語で）**：alice のクッキーを付けて、ノートを作る POST を4通り送ります。ヘッダーが何も無い場合、同じサイトのフォームから送った場合（ `Origin: http://127.0.0.1:3000` ）、ほかのサイトのフォームから送った場合（ `Origin: https://evil.example` ）、ブラウザが「ほかのサイトから」と印を付けた場合（ `Sec-Fetch-Site: cross-site` ）です。それぞれの返事と、最後に保存されたノートの数を表示します。

```rust
//! 第8巻 UNIT 01: ほかのサイトから送られてきたフォーム（CSRF）を、Topcoat が止めるかを確かめる。
//! 実行: cargo test --test u01_cross_site -- --nocapture

mod common;

use common::{TestApp, body_text, test_app};
use topcoat::router::{Body, Method, header, request::Request};

/// ログイン中のクッキーを付けて、ノートを作る POST を送る。`extra` のヘッダーも足す
async fn post_note(app: &TestApp, cookie: &str, extra: &[(&str, &str)]) -> (String,
String) {
    let mut builder = Request::builder()
        .method(Method::POST)
        .uri("/notes")
        .header(header::HOST, "127.0.0.1:3000")
        .header(header::CONTENT_TYPE, "application/x-www-form-urlencoded")
        .header(header::COOKIE, cookie);
    for (name, value) in extra {
        builder = builder.header(*name, *value);
    }
    let request = builder.body(Body::from("title=memo&body=")).unwrap();
    let response = app.router.handle(request).await;
    let status = response.status().to_string();
    (status, body_text(response).await)
}

#[tokio::test]
async fn cross_site_posts_are_rejected() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;

    let cases: [(&str, &[(&str, &str)]); 4] = [
        ("Origin も Sec-Fetch-Site も無い（curl など）", &[]),
        (
            "同じサイトのフォーム（Origin: http://127.0.0.1:3000）",
            &[("origin", "http://127.0.0.1:3000")],
        ),
        (
            "ほかのサイトのフォーム（Origin: https://evil.example）",
            &[("origin", "https://evil.example")],
        ),
        (
            "ほかのサイトから（Sec-Fetch-Site: cross-site）",
            &[("sec-fetch-site", "cross-site")],
        ),
    ];
    for (label, headers) in cases {
```

```
        let (status, body) = post_note(&app, &alice, headers).await;
        println!("{label}");
        println!(
            "    → {status}  本文: {:?}",
            body.chars().take(40).collect::<String>()
        );
    }

    // 実際に保存されたノートの数
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    let mut rows = conn
        .query("SELECT count(*) FROM notes", ())
        .await
        .expect("notes を読めません");
    let row = rows
        .next()
        .await
        .expect("行を読めません")
        .expect("行がありません");
    let count: i64 = row.get(0).expect("件数");
    println!("保存されたノート: {count} 件");
}
```

```
cargo test --test u01_cross_site -- --nocapture
```

▼ 実行結果

```
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.17s
     Running tests\u01_cross_site.rs (target\debug\deps\u01_cross_site-
1797bc835c7da007.exe)

running 1 test
POST /signup -> 303 See Other (443.652ms)
POST /notes -> 303 See Other (6.1553ms)
Origin も Sec-Fetch-Site も無い（curl など）
    → 303 See Other  本文: ""
POST /notes -> 303 See Other (6.8665ms)
同じサイトのフォーム（Origin: http://127.0.0.1:3000）
    → 303 See Other  本文: ""
ほかのサイトのフォーム（Origin: https://evil.example）
    → 403 Forbidden  本文: "forbidden"
ほかのサイトから（Sec-Fetch-Site: cross-site）
    → 403 Forbidden  本文: "forbidden"
保存されたノート: 2 件
test cross_site_posts_are_rejected ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.48s
```

▼ コードを1つずつ分解して解説

解説1: ほかのサイトからの POST は、403 で止まる

- `Origin: https://evil.example` と `Sec-Fetch-Site: cross-site` の2つは、`403 Forbidden`（本文は `forbidden`）でした。クッキーは正しいのに、断られています。保存されたノートは 2 件で、断られた2回の分は保存されていません。
- ブラウザは、フォームを送るとき、`Origin`（どのサイトのページから送ったか）と `Sec-Fetch-Site`（同じサイトからか、ほかのサイトからか）を、自動で付けます。ページの JavaScript が勝手に書き換えることはできません。Topcoat は、この2つを見て、ほかのサイトから来た「状態を変えるリクエスト」（GET・HEAD・OPTIONS 以外）を止めます。この守りは Topcoat に最初から入っていて、RustNote は何も書いていません。

解説2: ヘッダーが無いリクエストは、通る

- `Origin` も `Sec-Fetch-Site` も無いリクエスト（curl やテストの道具など）は、303 で通りました。CSRF は「ログイン中の人のブラウザに、勝手に送らせる」攻撃です。ブラウザは必ずこのヘッダーを付けるので、ヘッダーが無いリクエストは、ブラウザから来たものではありません。そうしたリクエストを送れるのは、クッキーを自分で持っている人だけです。
- 同じサイトのフォームから送った場合（`Origin: http://127.0.0.1:3000`）も通りました。`Origin` と、リクエストの `Host`（`127.0.0.1:3000`）が一致したからです。

解説3: 403 のときは、アクセスログに行が出ない

- 出力をよく見ると、303 の2回には `POST /notes -> 303 See Other (...)` のアクセスログがありますが、403 の2回にはありません。Topcoat の Origin の検査は、RustNote のアクセスログの層（ `routes.rs` の `access_log` ）よりも**前**で動くからです。攻撃を止めた記録を残したいときは、この点に気をつけます（UNIT 10（準備中））。

RustNote の脅威モデルの表

ここまでの実験をもとに、RustNote の脅威モデルを表にします。この表が、この巻の地図です。

攻める人と攻め方	狙う入口	RustNote の守り	確かめた所・くわしい単元
ほかの利用者が、URL の数字を変える（IDOR）	URL の ID	すべての SQL に `user_id = ?` 、他人のノートは 404	例1-1・エラー3・UNIT 07
ログインしていない人が、ノートの画面を開く	URL	`require_user` で 303、ログイン画面へ	例1-1・UNIT 05
ほかのサイトが、ブラウザにフォームを送らせる（CSRF）	フォーム・クッキー	Topcoat の Origin の検査で 403、`SameSite=Lax` 、状態を変える操作は POST だけ	例1-3・UNIT 06
ページに入れた悪い JavaScript が、クッキーを盗む	クッキー	`HttpOnly` （JavaScript から読めない）、`view!` の自動エスケープ	例1-2・UNIT 06
DB を盗んだ人が、パスワードとトークンを読む	（入口を通らない）	パスワードは Argon2id、トークンは SHA-256 のハッシュだけを保存	例1-2・UNIT 02・05
失敗の文の違いから、登録済みのメールを探る	ログインのフォーム	ログインは「メールアドレスまたはパスワードが違います」の同じ文、未登録でもダミーのハッシュを計算	UNIT 04
パスワードを何回も試す（総当たり）	ログインのフォーム	**守りが無い**（回数の制限が無い）	問1・UNIT 04
ログアウトした後の古いクッキーを使う	クッキー	ログアウトで `sessions` の行を消す	問2・UNIT 05

- 表の最後から2行目のように、**守りが無い**ことも、はっきり書きます。脅威モデルの目的は、「全部守れている」と言うことではなく、「どこが弱いかを、みんなが知っている」状態にすることです。
- 新規登録では、すでに登録されたメールアドレスを入れると「そのメールアドレスは既に登録されています」（409）と表示されます。使いやすさのためですが、登録済みかどうかが他人に分かる、という弱さでもあります。どちらを選ぶかは、UNIT 03（準備中）で考えます。

よくあるエラーと読み方

エラー1: 403 Forbidden（本文は forbidden だけ）

例1-3で、ほかのサイトから送ったときの返事です。自分でフォームを作ってブラウザで試したのに 403 になった、という場面でも出会います。

```
ほかのサイトのフォーム（Origin: https://evil.example）
    → 403 Forbidden  本文: "forbidden"
```

- **何が起きたか:** Topcoat の Origin の検査が、「このリクエストは、ほかのサイトのページから来た」と判断して止めました。本文は `forbidden` の1語だけで、RustNote のエラー画面も、アクセスログの行も出ません。
- **自分のサイトなのに 403 になるとき:** フォームのページを表示したサイトと、送り先の RustNote が、別のサイトとして扱われている場合です。たとえば、フォームだけを書いた HTML ファイルを別の開発用のサーバー（別のポート）で開いて、そこから RustNote へ送ると、`Origin` と `Host` のポートが違うので止まります。また、RustNote の手前に置いた中継のサーバー（リバースプロキシ）が `Host` を書き換える設定だと、同じサイトからでも食い違います。フォームは RustNote 自身の画面から送り、中継のサーバーは `Host` をそのまま渡す設定にします。ほかのサイトからの送信をわざと受け付けたいときは、Topcoat の `OriginPolicy` で、信じてよい送り元を指定できます。守りそのものを外す設定もありますが、使わないでください。

エラー2: 持ち主を渡さずにノートを探そうとした（E0308・E0061）

RustNote のリポジトリを使って、ノートを ID だけで探そうとした場合です（`examples/e01_find_without_owner.rs` に保存して試します）。

```rust
use rustnote::db::{self, Db};
use rustnote::domain::NoteId;
use rustnote::state::AppState;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("e01.db")).await?;
    db::migrate(&db).await?;
    let state = AppState::new(db);
    // 持ち主（owner）を渡さずに、ノートを ID だけで探そうとした
    let note = state.notes.find(NoteId::new(1)?).await?;
    println!("{note:?}");
    Ok(())
}
```

```
cargo build --example e01_find_without_owner
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0308]: `?` operator has incompatible types
  --> examples\e01_find_without_owner.rs:12:33
   |
12 |     let note = state.notes.find(NoteId::new(1)?).await?;
   |                                 ^^^^^^^^^^^^^^^ expected `UserId`, found `NoteId`
   |
   = note: `?` operator cannot convert from `NoteId` to `UserId`

error[E0061]: this method takes 2 arguments but 1 argument was supplied
  --> examples\e01_find_without_owner.rs:12:28
   |
12 |     let note = state.notes.find(NoteId::new(1)?).await?;
   |                            ^^^^----------------- argument #2 of type `NoteId` is
missing
   |
note: method defined here
  --> src\repository\notes.rs:21:14
   |
21 |     async fn find(&self, owner: UserId, id: NoteId) -> Result<Option<Note>,
RepoError>;
   |              ^^^^
help: provide the argument
   |
12 |     let note = state.notes.find(NoteId::new(1)?, /* NoteId */).await?;
   |                                                +++++++++++++

Some errors have detailed explanations: E0061, E0308.
For more information about an error, try `rustc --explain E0061`.
error: could not compile `rustnote` (example "e01_find_without_owner") due to 2
previous errors
```

- **何が起きたか:** エラーが2つ出ました。1つ目の `E0308` は「`UserId` が来るはずの場所に、`NoteId` が来ました」です。`find` の1つ目の引数は持ち主（`owner: UserId`）なので、ノートの ID を渡すと型が合いません。2つ目の `E0061` は「このメソッドは引数を2つとるのに、1つしか渡されていません」です。`note:` が、`src/repository/notes.rs` の 21 行目の `async fn find(&self, owner: UserId, id: NoteId)` を指しています。
- **これはエラーで正しい:** RustNote は、ノートを読む・変えるメソッドに、必ず持ち主を渡すように作っています。持ち主の確かめを忘れると、コンパイルが通りません。しかも `UserId` と `NoteId` は別の型（newtype）なので、2つを入れ替えてもエラーになります（第4巻 UNIT 01）。
- **help の案内に注意:** `help: provide the argument` は、`find(NoteId::new(1)?, /* NoteId */)` のように、足りない2つ目を足す形を案内しています。コンパイラーは「並び順のまちがい」までは分からないので、案内のとおりに直すと、まだ1つ目の型が合いません。正しくは、ログイン中のユーザーの ID を1つ目に渡して `find(user.id, note_id)` とします。確かめたら、このファイルは消しておきます。

エラー3（エラーが出ない間違い）: SQL に持ち主の条件を書き忘れた

リポジトリを通さずに、SQL を自分で書いたときに起きやすい間違いです。alice のノートを、bob の立場で探します（`examples/u01_idor.rs`）。

```rust
//! 認可の確かめを忘れた SQL（持ち主の条件が無い）と、RustNote と同じ SQL を比べる。
//! 実行: cargo run -q --example u01_idor

use rustnote::db::{self, Db};

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let dir = tempfile::tempdir()?;
    let db = Db::open_local(dir.path().join("u01.db")).await?;
    db::migrate(&db).await?;
    let conn = db.connect().await?;
    conn.execute_batch(
        "INSERT INTO users (email, password_hash)
             VALUES ('alice@example.com', 'x'), ('bob@example.com', 'x');
         INSERT INTO notes (user_id, title) VALUES (1, 'alice の秘密のメモ');",
    )
    .await?;

    // bob（user_id = 2）が、URL の /notes/1 を開いたとする
    let bob: i64 = 2;
    let note_id: i64 = 1;

    // 間違い: ID だけで探す（「この人が見てよいか」を確かめていない）
    let mut rows = conn
        .query("SELECT title FROM notes WHERE id = ?1", [note_id])
        .await?;
    match rows.next().await? {
        Some(row) => println!("ID だけで探す: 見つかった → {}", row.get::<String>(0)?),
        None => println!("ID だけで探す: 見つからない"),
    }

    // RustNote と同じ: 持ち主の条件も付けて探す
    let mut rows = conn
        .query(
            "SELECT title FROM notes WHERE id = ?1 AND user_id = ?2",
            [note_id, bob],
        )
        .await?;
    match rows.next().await? {
        Some(row) => println!(
            "持ち主の条件も付ける: 見つかった → {}",
            row.get::<String>(0)?
        ),
        None => println!("持ち主の条件も付ける: 見つからない（404 にする）"),
    }
    Ok(())
}
```

```
cargo run -q --example u01_idor
```

```
ID だけで探す: 見つかった → alice の秘密のメモ
持ち主の条件も付ける: 見つからない（404 にする）
```

- **何が起きたか:** ID だけで探すと、bob に alice のノートが見つかってしまいました。コンパイルも通り、エラーも出ません。画面の処理がこの SQL を使っていたら、bob は URL の数字を 1, 2, 3, … と変えていくだけで、全員のノートを読めてしまいます。これが **IDOR**（Insecure Direct Object Reference：他人のデータを、ID を変えるだけで直接見られてしまう弱さ）です。
- **直し方:** 持ち主の条件 `AND user_id = ?2` を付けます。RustNote のリポジトリは、すべての SQL にこの条件を入れています。そして、エラー2のとおり、リポジトリのメソッドは持ち主を受け取らないと呼べません。**SQL を書く場所をリポジトリに集め、持ち主を必ず受け取る形にする**のが、IDOR を防ぐいちばん確実な方法です（第9巻 UNIT 04）。

RustNote ではここで使う

ログインとログアウトは、`src/auth.rs` の2つの関数です（抜き出しています）。

```
/// ログインさせる: 新しいトークンを発行してクッキーに入れ、そのハッシュを DB に記録する
pub async fn log_in(cx: &Cx, user: &User) -> Result<(), AppError> {
    let state = app_state(cx);
    // 前のセッションのクッキーが残っていれば、その記録は先に消しておく
    if let Some(old_hash) = session::token_hash(cx).await.map_err(AppError::internal)?
{
        state.sessions.delete(&old_hash).await?;
    }
    // 毎回新しいトークンを作る（セッション固定攻撃を防ぐ）
    let session = session::start(cx).await.map_err(AppError::internal)?;
    state
        .sessions
        .create(&session.token_hash, user.id, session.expires_at)
        .await?;
    // ついでに、期限切れのセッションを掃除しておく
    state.sessions.delete_expired(SystemTime::now()).await?;
    Ok(())
}

/// ログアウトさせる: クッキーを消し、DB の記録も消す
pub async fn log_out(cx: &Cx) -> Result<(), AppError> {
    if let Some(token_hash) = session::stop(cx).await.map_err(AppError::internal)? {
        app_state(cx).sessions.delete(&token_hash).await?;
    }
    Ok(())
}
```

- `session::start(cx)` は、Topcoat が新しいトークンを作ってクッキーに入れ、そのハッシュ（`token_hash`）と期限（`expires_at`）を返す関数です。Topcoat は、トークンを**発行するだけ**で、保存はしません。保存するのはアプリ（RustNote の `sessions` 表）の役目です。
- ログインのたびに**新しい**トークンを作るのは、「攻める人が先に用意したトークンを、本人にログインさせて使わせる」攻撃（セッション固定攻撃）を防ぐためです。くわしくは UNIT 05（準備中）で扱います。
- `log_out` は、`session::stop(cx)` でクッキーを消し、返ってきたハッシュで `sessions` の行も消します。どちらか片方だけでは足りない理由は、問2で確かめます。

ログインの失敗は、`src/routes/login.rs` で、理由を区別せずに同じ文で返しています（抜き出しています）。

```
const LOGIN_FAILED: &str = "メールアドレスまたはパスワードが違います";
```

```
    // メールが無いのか、パスワードが違うのかは区別しない
    let error = Some(LOGIN_FAILED.to_string());
    Ok(view! {
        (StatusCode::UNAUTHORIZED)
        login_form(email: input.email, error: error)
    })
```

練習問題

問1: 脅威モデルの表で「守りが無い」とした、パスワードの総当たりを確かめます。alice を登録してから、まちがったパスワードで 10 回続けてログインし、11 回目に正しいパスワードを送るテストを書いてください。RustNote は、何回目かで止めるでしょうか。

問2: ログアウトしたとき、DB の `sessions` の行と、ブラウザのクッキーはどうなるでしょうか。テストで、ログアウトの前と後の `sessions` の行の数、ログアウトの返事の `Set-Cookie`、ログアウト前のクッキーでもう一度 `/notes` を開いた結果を表示してください。

問3: もし RustNote が、`sessions` 表に、トークンのハッシュではなく**トークンそのもの**を保存していたら、DB のファイルが盗まれたときに何が起きるでしょうか。例1-2の結果をもとに説明してください。

解答と解説

問1の解答:

`tests/u01_q1.rs` です。

```rust
//! 問1の解答: パスワードを何回まちがえても、RustNote は止めない（回数の制限が無い）ことを確かめる。
//! 実行: cargo test --test u01_q1 -- --nocapture

mod common;

use std::time::Instant;

use common::test_app;

#[tokio::test]
async fn many_wrong_logins_are_not_limited() {
    let app = test_app().await;
    app.sign_up("alice@example.com", "correct horse battery")
        .await;

    let started = Instant::now();
    let mut statuses = Vec::new();
    for _ in 0..10 {
        let response = app
            .post_form(
                "/login",
                "email=alice%40example.com&password=wrong-password",
                None,
            )
            .await;
        statuses.push(response.status().as_u16());
    }
    println!("10 回まちがえた結果: {statuses:?}");
    println!("10 回にかかった時間: {:.1?}", started.elapsed());

    // 11 回目に、正しいパスワードを送る
    let response = app
        .post_form(
            "/login",
            "email=alice%40example.com&password=correct+horse+battery",
            None,
        )
        .await;
    println!("11 回目に正しいパスワード → {}", response.status());
}
```

```
cargo test --test u01_q1 -- --nocapture
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.24s
     Running tests\u01_q1.rs (target\debug\deps\u01_q1-81fa10c5a5eabbd3.exe)

running 1 test
POST /signup -> 303 See Other (411.0828ms)
POST /login -> 401 Unauthorized (398.3169ms)
POST /login -> 401 Unauthorized (390.2362ms)
POST /login -> 401 Unauthorized (388.3874ms)
POST /login -> 401 Unauthorized (374.6642ms)
POST /login -> 401 Unauthorized (380.8843ms)
POST /login -> 401 Unauthorized (385.9132ms)
POST /login -> 401 Unauthorized (375.7783ms)
POST /login -> 401 Unauthorized (425.9443ms)
POST /login -> 401 Unauthorized (398.9875ms)
POST /login -> 401 Unauthorized (399.0318ms)
10 回まちがえた結果: [401, 401, 401, 401, 401, 401, 401, 401, 401, 401]
10 回にかかった時間: 3.9s
POST /login -> 303 See Other (413.2169ms)
11 回目に正しいパスワード → 303 See Other
test many_wrong_logins_are_not_limited ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
4.78s
```

- 10 回とも `401 Unauthorized` で、11 回目の正しいパスワードは `303 See Other`（ログイン成功）でした。RustNote は、何回まちがえても止めません。脅威モデルの表のとおり、**回数の制限が無い**ことが確かめられました。
- 1回のまちがいに、約 0.4 秒かかっています。これは debug ビルドで Argon2id の照合をしている時間で、release ビルドでは約 20 ms です（第12巻 UNIT 01）。重い照合は総当たりを遅くしますが、止めはしません。たとえば 1 秒に 50 回なら、1 日で 400 万回以上試せてしまいます。
- 直すには、「同じメールアドレスへの失敗が、10 分に 5 回を超えたら、しばらく断る」「同じ送り元からの回数を数える」といった**回数の制限**が必要です。どこに、どの単位で数えるかは、UNIT 04（準備中）で扱います（RustNote のコードは、この巻では変えません）。

問2の解答:

`tests/u01_q2.rs` です。

```rust
//! 問2の解答: ログアウトすると、DB の sessions の行と、ブラウザのクッキーはどうなるか。
//! 実行: cargo test --test u01_q2 -- --nocapture

mod common;

use common::{TestApp, location, test_app};
use topcoat::router::header;

/// sessions 表の行の数
async fn sessions(app: &TestApp) -> i64 {
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    let mut rows = conn
        .query("SELECT count(*) FROM sessions", ())
        .await
        .expect("sessions を読めません");
    let row = rows
        .next()
        .await
        .expect("行を読めません")
        .expect("行がありません");
    row.get(0).expect("件数")
}

#[tokio::test]
async fn logout_deletes_the_session_record() {
    let app = test_app().await;
    let alice = app
        .sign_up("alice@example.com", "correct horse battery")
        .await;
    println!("ログイン中の sessions の行: {} 件", sessions(&app).await);

    let response = app.post_form("/logout", "", Some(&alice)).await;
    let cleared = response
        .headers()
        .get_all(header::SET_COOKIE)
        .iter()
        .filter_map(|value| value.to_str().ok())
        .find(|value| value.starts_with("__Host-session="))
        .unwrap_or("（無し）")
        .to_string();
    println!("ログアウトの返事: {}", response.status());
    println!("消すためのクッキー: {cleared}");
    println!("ログアウト後の sessions の行: {} 件", sessions(&app).await);

    // ログアウト前のクッキーを、もう一度見せる
    let response = app.get("/notes", Some(&alice)).await;
    println!(
        "古いクッキーで /notes → {}（移動先 {}）",
        response.status(),
```

```
        location(&response)
    );
}
```

```
cargo test --test u01_q2 -- --nocapture
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.46s
     Running tests\u01_q2.rs (target\debug\deps\u01_q2-d924c1589f8ead12.exe)

running 1 test
POST /signup -> 303 See Other (433.8285ms)
ログイン中の sessions の行: 1 件
POST /logout -> 303 See Other (12.6138ms)
ログアウトの返事: 303 See Other
消すためのクッキー: __Host-session=; HttpOnly; SameSite=Lax; Secure; Path=/; Max-Age=0;
Expires=Mon, 27 Oct 2025 15:04:10 GMT
ログアウト後の sessions の行: 0 件
GET /notes -> 303 See Other (2.5015ms)
古いクッキーで /notes → 303 See Other（移動先 /login）
test logout_deletes_the_session_record ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.48s
```

- ログアウトで、`sessions` の行は 1 件から 0 件になりました。返事の `Set-Cookie` は、値が空の `__Host-session=` と、`Max-Age=0`・過去の `Expires`（1年前）で、ブラウザに「このクッキーを今すぐ消せ」と伝えています。
- ログアウト前のクッキーをもう一度見せると、303 でログイン画面へ移りました。クッキーのトークンをハッシュにしても、`sessions` 表に同じ行が無いからです。
- **片方だけでは足りない理由:** ブラウザのクッキーを消すだけ（DB の行を残す）だと、ログアウトの前にトークンを盗まれていた場合、盗んだ人は、そのトークンで 30 日間ログインしたままでいられます。DB の行を消すことで、**サーバーの側で**そのトークンを使えなくするのです。ブラウザはクッキーを消すよう頼まれても、従わないこと（壊れたブラウザや、攻める人の道具）もあります。

問3の解答:

- 例1-2で見たとおり、クッキーに入るのはトークンそのもの（46 文字）です。もし `sessions` 表にトークンそのものが入っていたら、DB を盗んだ人は、表から値を取り出して、自分のブラウザのクッキー `__Host-session` に入れるだけで、**期限の切れていない全員に、パスワードを知らないまま、なりすませます。**
- 実際の RustNote は、トークンの SHA-256（`token_hash`）だけを保存しています。SHA-256 は1方向にしか計算できないので、ハッシュからトークンを作り直すことはできません。トークンは十分に長いランダムな値なので、総当たりで同じハッシュになる値を探すこともできません。

- パスワードを Argon2id で、トークンを SHA-256 で、と使い分けているのは、パスワードは人が覚えられる短い値なので総当たりされやすく、わざと重い計算が必要だからです。トークンはもともと長いランダムな値なので、軽い SHA-256 で十分です（UNIT 02・05（準備中））。

まとめ

- 認証（誰か）・セッション（同じ人か）・認可（してよいか）は別々の確かめ。RustNote は、認証が無ければ 303 でログイン画面へ、持ち主でなければ 404 にする。
- ブラウザにはトークン、DB にはそのハッシュだけを置く。クッキーは `__Host-` の名前と `HttpOnly`・`Secure`・`SameSite=Lax` で守り、ほかのサイトからの POST は Topcoat の Origin の検査が 403 で止める。
- 脅威モデルで、守るもの・攻める人・入口・信頼の境目を書き出し、守りが無い所（RustNote ではログインの回数の制限）も、はっきり表に残す。

UNIT 02: パスワード・Argon2id・salt

この単元のゴール

- パスワードを「暗号化」ではなく「ハッシュ」にして保存する理由と、salt（毎回ちがう、ランダムな値）の役目を説明できる
- RustNote が DB に保存する PHC 文字列（`$argon2id$v=19$m=19456,t=2,p=1$…`）を、部分ごとに読める
- 速いハッシュと Argon2id の速さの違いを実際に測り、パスワードに「わざと重い」ハッシュを使う理由を説明できる
- 保存したハッシュが壊れていたとき・古い資料の書き方を写したときのエラーを読み、ログインの失敗の時間の差（DUMMY_HASH）を確かめられる

たとえ話でつかむ

図書館の会員カードの、暗証番号を預かる場面を考えます。

- 暗証番号を紙に**そのまま**書いて金庫にしまうと、金庫を開けられた日に、全員の暗証番号が読まれてしまいます。
- そこで、暗証番号を**ミキサー**にかけます。ミキサーは、同じ材料なら必ず同じジュースになりますが、ジュースから材料に戻すことはできません。金庫には、ジュースだけをしまいます。会員が来たら、その場で暗証番号をミキサーにかけて、金庫のジュースと同じかを比べます。
- ただし、「1234 のジュースはこの色」という**早見表**を作られると、ジュースから暗証番号が分かってしまいます。そこで、ミキサーにかけるたびに、**会員ごとに違う隠し味**（salt）を入れ、隠し味もジュースの横に書いておきます。同じ暗証番号でも、会員ごとにジュースが違うので、早見表が使えません。
- さらに、ミキサーを**わざとゆっくり**回します。1杯作るのに時間がかかれば、金庫を盗んだ人が「0000 から 9999 まで全部試す」のに、とても長い時間がかかります。

図書館の暗証番号	RustNote のパスワード
紙にそのまま書く	平文（へいぶん）で保存する。**RustNote はしない**
ミキサー（戻せない）	**ハッシュ**（Argon2id）。ハッシュから元のパスワードには戻せない
会員ごとの隠し味	**salt**。ハッシュにするたびに、ランダムに作る 16 バイトの値
ジュースと隠し味を並べて書く	**PHC 文字列**（方式・設定・salt・ハッシュを、`$` で区切って1つの文字列にしたもの）
わざとゆっくり回す	Argon2id の**重さの設定**（`m=19456,t=2,p=1`）
早見表	前もって計算したハッシュの一覧（レインボーテーブルなど）。salt で使えなくなる

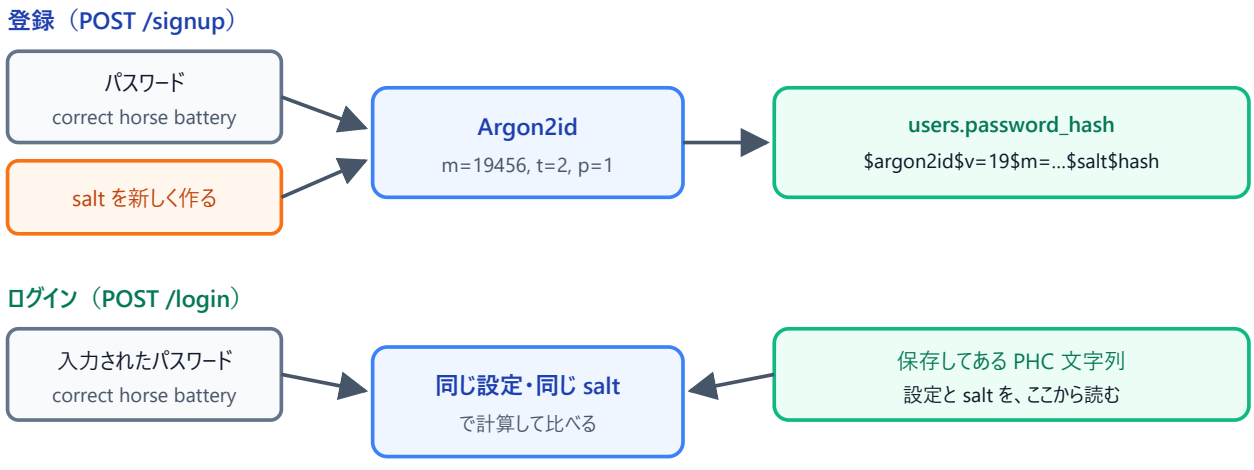
> **用語:** ハッシュ（hash：どんな長さのデータからでも、決まった長さの値を作る計算。同じ入力からは同じ値が出るが、値から入力には戻せない）

> **用語:** salt（ソルト：ハッシュにする前に、パスワードに混ぜるランダムな値。人ごと・回ごとに違う。秘密にする必要はなく、ハッシュと一緒に保存する）

> **用語:** 暗号化（あんごうか：鍵があれば元に戻せる変換。パスワードの保存には使わない。鍵を盗まれたら、全員のパスワードが戻されてしまうから）

図で理解する

登録のときと、ログインのときに、RustNote の中で起きることです。



同じなら true（ログイン成功）、違えば false。パスワードそのものは、どこにも保存しない

図8-5: パスワードのハッシュの流れ。登録のときは新しい salt で計算して PHC 文字列を保存し、ログインのときは、保存した文字列から設定と salt を読んで、同じ計算をして比べる

- **登録のとき**は、パスワードと、新しく作った salt を Argon2id に入れて、PHC 文字列を作り、`users.password_hash` に保存します。
- **ログインのとき**は、保存してある PHC 文字列から、設定（`m`・`t`・`p`）と salt を読み、入力されたパスワードを**同じ設定・同じ salt** で計算して、ハッシュが同じかを比べます。元のパスワードに戻して比べるのではありません。
- だから、DB の中にも、ログにも、パスワードそのものはどこにも残りません。

PHC 文字列の中身は、次のように並んでいます。

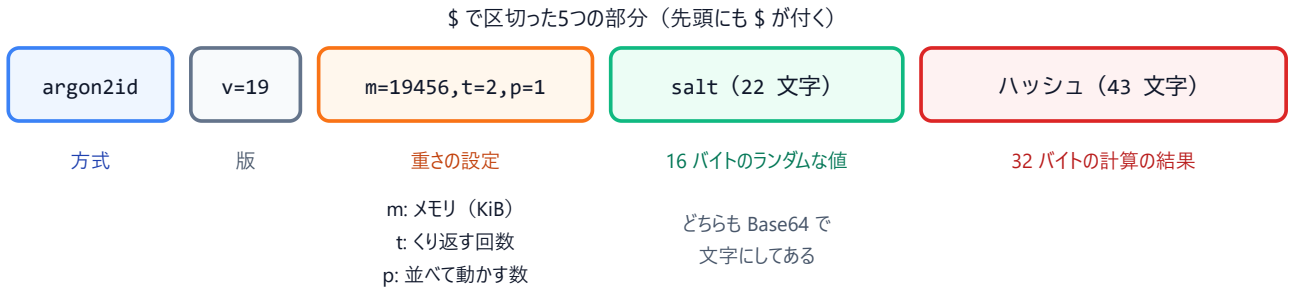


図8-6: PHC 文字列の部分。方式・版・重さの設定・salt・ハッシュが、1つの文字列に入っているので、照合に必要なものは、この文字列だけで足りる

まず動かす最小コード

例2-1: 同じパスワードを、2回ハッシュにする

> **▼ このコードがやること（先に日本語で）**: RustNote の `auth::hash_password` で、同じパスワードを2回ハッシュにして、2つの PHC 文字列を表示し、同じかどうかを比べます。1つ目を `$` で分けて、中身を1つずつ表示します。最後に `auth::verify_password` で、正しいパスワードと1文字違うパスワードを照合します。

```rust
//! RustNote の hash_password で、同じパスワードを2回ハッシュにして比べる。
//! PHC 文字列（$argon2id$...）を「$」で分けて、中身を1つずつ表示する。
//! 実行: cargo run -q --example u02_hash

use rustnote::auth::{hash_password, verify_password};
use rustnote::domain::Password;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let password = Password::parse("correct horse battery")?;
    let first = hash_password(&password).await?;
    let second = hash_password(&password).await?;

    println!("1回目: {first}");
    println!("2回目: {second}");
    println!("2つは同じか: {}", first == second);

    // $argon2id$v=19$m=19456,t=2,p=1$<salt>$<hash> を「$」で分ける
    let parts: Vec<&str> = first.split('$').collect();
    println!("--- 1回目を「$」で分ける");
    println!("方式: {}", parts[1]);
    println!("版: {}", parts[2]);
    println!("重さの設定: {}", parts[3]);
    println!("salt（{} 文字）: {}", parts[4].len(), parts[4]);
    println!("ハッシュ（{} 文字）: {}", parts[5].len(), parts[5]);

    println!("--- 照合");
    println!(
        "正しいパスワード × 1回目: {}",
        verify_password("correct horse battery", &first).await?
    );
    println!(
        "正しいパスワード × 2回目: {}",
        verify_password("correct horse battery", &second).await?
    );
    println!(
        "1文字違うパスワード × 1回目: {}",
        verify_password("correct horse batterY", &first).await?
    );
    Ok(())
}
```

コピーした `rustnote` フォルダの `examples/u02_hash.rs` として保存して実行します。

```
cargo run -q --example u02_hash
```

▼ **実行結果**（salt とハッシュは、実行するたびに変わります）

```
1回目:
$argon2id$v=19$m=19456,t=2,p=1$/gnrD6uzcYJQwvdc4zRYaA$q7m7sj84y3lpN7nLk5xPkVCkU9jY1wuYJ
Qn6LWEPunw
2回目:
$argon2id$v=19$m=19456,t=2,p=1$TJWrC9mQPy8CJEmapKizxw$48kU8XwN/fntjhy815yrZtkaPfyIiigql
3DzWNrIzbE
2つは同じか: false
--- 1回目を「$」で分ける
方式: argon2id
版: v=19
重さの設定: m=19456,t=2,p=1
salt（22 文字）: /gnrD6uzcYJQwvdc4zRYaA
ハッシュ（43 文字）: q7m7sj84y3lpN7nLk5xPkVCkU9jY1wuYJQn6LWEPunw
--- 照合
正しいパスワード × 1回目: true
正しいパスワード × 2回目: true
1文字違うパスワード × 1回目: false
```

▼ コードを1つずつ分解して解説

解説1: 同じパスワードでも、ハッシュは毎回違う

- 1回目と2回目は、同じ `correct horse battery` なのに、違う文字列になりました（`2つは同じか: false`）。違うのは、後ろの2つの部分（salt とハッシュ）だけです。`hash_password` が、呼ばれるたびに新しい salt を作っているからです。
- それでも、照合はどちらも `true` です。照合のときは、それぞれの文字列に書いてある salt を使って計算し直すからです。

解説2: 文字列の中に、照合に必要なものが全部ある

- `$` で分けると、方式（`argon2id`）・版（`v=19`）・重さの設定（`m=19456,t=2,p=1`）・salt（22 文字）・ハッシュ（43 文字）の5つになりました。salt は 16 バイト、ハッシュは 32 バイトの値を、Base64（英数字と `+`・`/` で表す書き方）で文字にしたものです。
- 設定まで一緒に保存しておくので、あとで設定を重くしても、古い設定で作ったハッシュを照合できます。古いハッシュは、その人が次にログインして照合に成功したときに、新しい設定で作り直せばよいのです。

解説3: 1文字違えば、まったく違う結果になる

- 最後の `y` を `Y` にしただけで、照合は `false` になりました。ハッシュは、入力が少しでも違えば、まったく違う値になるように作られています。「だいたい合っている」はありません。

もう一歩: 実務に近い形

例2-2: 速いハッシュと Argon2id は、1秒に何回計算できるか

ハッシュなら何でもよいわけではありません。パスワードには、**わざと重い**ハッシュを使います。どれくらい違うのかを測ります。

▼ **このコードがやること（先に日本語で）**：1秒のあいだに、パスワードの候補（ `password0`・`password1`・…）を作ってはハッシュにする、をくり返し、何回できたかを数えます。標準ライブラリの速いハッシュ（ `DefaultHasher` ）と、RustNote と同じ `Argon2::default()` で比べます。速さを測るので、release ビルドで動かします（第12巻 UNIT 01）。

```rust
//! 1秒のあいだに、何回ハッシュを計算できるかを比べる。
//! 速いハッシュ（標準ライブラリの DefaultHasher）と、Argon2id（RustNote と同じ既定の設定）。
//! release で動かす: cargo run -q --release --example u02_speed

use std::hash::{DefaultHasher, Hash, Hasher};
use std::hint::black_box;
use std::time::{Duration, Instant};

use argon2::{Argon2, password_hash::PasswordHasher};

/// 1秒のあいだに `work` を何回くり返せるかを数える
fn per_second(mut work: impl FnMut(u64)) -> u64 {
    let started = Instant::now();
    let mut count = 0;
    while started.elapsed() < Duration::from_secs(1) {
        work(count);
        count += 1;
    }
    count
}

fn main() {
    // 速いハッシュ: パスワードの候補を作っては、ハッシュにする
    let fast = per_second(|n| {
        let mut hasher = DefaultHasher::new();
        format!("password{n}").hash(&mut hasher);
        black_box(hasher.finish());
    });
    println!("DefaultHasher（速いハッシュ）: 1秒に {fast} 回");

    // Argon2id: RustNote と同じ Argon2::default()（m=19456, t=2, p=1）
    let argon2 = Argon2::default();
    let slow = per_second(|n| {
        let candidate = format!("password{n}");
        black_box(
            argon2
                .hash_password(candidate.as_bytes())
                .expect("ハッシュにできません"),
        );
    });
    println!("Argon2id（わざと重いハッシュ）: 1秒に {slow} 回");
    println!("差: 約 {} 倍", fast / slow.max(1));
}
```

```
cargo run -q --release --example u02_speed
```

▼ 実行結果

```
DefaultHasher（速いハッシュ）： 1秒に 8721779 回
Argon2id（わざと重いハッシュ）： 1秒に 39 回
差: 約 223635 倍
```

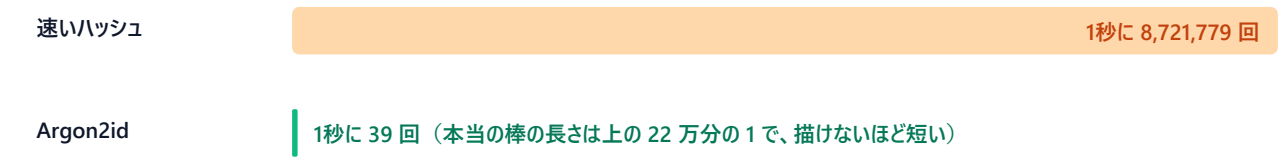


図8-7: 1秒に計算できる回数。この PC（Windows ARM64、release ビルド、1つのスレッド）で実際に測った数。わざと重い Argon2id は、総当たりを何十万倍も遅くする

- 速いハッシュは1秒に約 870 万回、Argon2id は 39 回でした。約 22 万倍の差です。
- 1回のログインで 1 回計算するだけなら、Argon2id の 1/39 秒（約 26 ミリ秒）は、利用者が待つほどの時間ではありません。ところが、DB を盗んだ人が、よく使われるパスワードを 1 億通り試すとすると、速いハッシュなら約 11 秒、Argon2id なら約 30 日かかります（この PC 1台で、1つのスレッドで計算した場合の見積もりです）。**正しい人には気にならず、総当たりする人にはとても重い**のが、パスワード用のハッシュの役目です。
- Argon2id は、計算に毎回 19 MiB（`m=19456` KiB）のメモリを使います。メモリを多く使う計算は、たくさんの計算を同時に進める専用の機械（GPU など）でも、速くしにくいように作られています。
- `DefaultHasher` は、`HashMap` の表を作るためのハッシュで、結果も 64 ビットと短く、パスワードのような秘密を守るためのものではありません。ここでは「速いハッシュ」の例として使っています。SHA-256 のような安全なハッシュでも、速すぎるので、パスワードの保存には向きません。

> **注意:** 速さは、PC や設定によって大きく変わります。この例の数は、この PC で測った目安です。重さの設定は、OWASP（Web の安全についての資料をまとめている団体）が勧める Argon2id の最低の設定（m=19456, t=2, p=1）と同じで、argon2 0.6 の `Argon2::default()` が使う値です。

例2-3: DB には、何が保存されているか

> **▼ このコードがやること（先に日本語で）**: 1つ目のテストでは、alice と bob を**同じパスワード**で登録し、`users` 表の `password_hash` の先頭と長さ、パスワードそのものが入っていないか、2人のハッシュが同じかを表示します。2つ目のテストでは、alice のハッシュを、読めない文字列（`broken`）に壊してから、正しいパスワードでログインし、どうなるかを表示します。

```rust
//! 第8巻 UNIT 02: DB の users 表に、パスワードがどんな形で保存されるかを確かめる。
//! 実行: cargo test --test u02_stored -- --nocapture --test-threads=1

mod common;

use common::{TestApp, body_text, test_app};

const PASSWORD: &str = "correct horse battery";

/// users 表の、メールアドレスとパスワードのハッシュを読む
async fn stored(app: &TestApp) -> Vec<(String, String)> {
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    let mut rows = conn
        .query("SELECT email, password_hash FROM users ORDER BY id", ())
        .await
        .expect("users を読めません");
    let mut users = Vec::new();
    while let Some(row) = rows.next().await.expect("行を読めません") {
        users.push((
            row.get(0).expect("email"),
            row.get(1).expect("password_hash"),
        ));
    }
    users
}

#[tokio::test]
async fn same_password_different_hashes() {
    let app = test_app().await;
    app.sign_up("alice@example.com", PASSWORD).await;
    app.sign_up("bob@example.com", PASSWORD).await;

    let users = stored(&app).await;
    for (email, hash) in &users {
        println!("{email}: {}… ({} 文字) ", &hash[..31], hash.len());
        println!(
            "    パスワードそのものが入っている: {}",
            hash.contains(PASSWORD)
        );
    }
    println!(
        "alice と bob のハッシュは同じか: {}",
        users[0].1 == users[1].1
    );
}

#[tokio::test]
async fn broken_hash_makes_login_fail_safely() {
    let app = test_app().await;
```

```
    app.sign_up("alice@example.com", PASSWORD).await;

    // DB の中のハッシュを、読めない文字列に壊してしまったとする
    let conn = app.db.db.connect().await.expect("DB に接続できません");
    conn.execute(
        "UPDATE users SET password_hash = 'broken' WHERE email = 'alice@example.com'",
        (),
    )
    .await
    .expect("更新できません");

    let form = "email=alice%40example.com&password=correct+horse+battery";
    let response = app.post_form("/login", form, None).await;
    let status = response.status();
    let html = body_text(response).await;
    println!("壊れたハッシュでログイン → {status}");
    println!(
        "画面に「時間をおいてもう一度お試しください」: {}",
        html.contains("時間をおいてもう一度お試しください")
    );
}
```

```
cargo test --test u02_stored -- --nocapture --test-threads=1
```

▼ 実行結果

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 6.76s
     Running tests\u02_stored.rs (target\debug\deps\u02_stored-64195f227e1b0a63.exe)

running 2 tests
test broken_hash_makes_login_fail_safely ... POST /signup -> 303 See Other (418.678ms)
[error] サーバーの中でエラーが起きました <- invalid encoding
POST /login -> 500 Internal Server Error (8.9707ms)
壊れたハッシュでログイン → 500 Internal Server Error
画面に「時間をおいてもう一度お試しください」: true
ok
test same_password_different_hashes ... POST /signup -> 303 See Other (419.3062ms)
POST /signup -> 303 See Other (426.7911ms)
alice@example.com: $argon2id$v=19$m=19456,t=2,p=1$…（97 文字）
    パスワードそのものが入っている: false
bob@example.com: $argon2id$v=19$m=19456,t=2,p=1$…（97 文字）
    パスワードそのものが入っている: false
alice と bob のハッシュは同じか: false
ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
1.34s
```

- 2人とも、`$argon2id$v=19$m=19456,t=2,p=1$` で始まる 97 文字の文字列で、パスワードそのものは入っていません。同じパスワードなのに、2人のハッシュは違います（salt が違うから）。DB を見ても、「alice と bob は同じパスワードだ」とは分かりません。
- ハッシュを壊すと、正しいパスワードでもログインは `500 Internal Server Error` になり、画面には「時間をおいてもう一度お試しください」だけが出ました。サーバーのログには `[error] サーバーの中でエラーが起きました <- invalid encoding`（文字列の形が正しくない）と、原因が残ります。「パスワードが違います」（401）ではなく 500 にしているのは、これが利用者のまちがいではなく、**DB の中身が壊れている**というサーバー側の問題だからです（`src/auth.rs` の `verify_password` は、保存した文字列を読めないときを、内部のエラーとして扱います）。

よくあるエラーと読み方

エラー1: 古い資料（argon2 0.5）の書き方を写した（E0432・E0061）

インターネットの記事には、argon2 の古い版（0.5）の書き方がたくさん残っています。それを、0.6 を使う RustNote にそのまま写した場合です（`examples/e02_old_api.rs`）。

```
// 古い資料（argon2 0.5 の頃）の書き方を、そのまま写してしまった
use argon2::password_hash::{SaltString, rand_core::OsRng};
use argon2::{Argon2, PasswordHasher};

fn main() {
    let salt = SaltString::generate(&mut OsRng);
    let hash = Argon2::default()
        .hash_password(b"correct horse battery", &salt)
        .expect("ハッシュにできません");
    println!("{hash}");
}
```

```
cargo build --example e02_old_api
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error[E0432]: unresolved imports `argon2::password_hash::SaltString`,
`argon2::password_hash::rand_core`
  --> examples\e02_old_api.rs:2:29
   |
 2 | use argon2::password_hash::{SaltString, rand_core::OsRng};
   |                             ^^^^^^^^^^  ^^^^^^^^^ could not find `rand_core` in
`password_hash`
   |                             |
   |                             no `SaltString` in the root
   |
note: found an item that was configured out
  --> ...\lib.rs:40:9
   |
39 | #[cfg(feature = "rand_core")]
   |       ---------------------- the item is gated behind the `rand_core` feature
40 | pub use rand_core;
   |         ^^^^^^^^^

error[E0061]: this method takes 1 argument but 2 arguments were supplied
   --> examples\e02_old_api.rs:8:10
    |
  8 |         .hash_password(b"correct horse battery", &salt)
    |          ^^^^^^^^^^^^^                           ----- unexpected argument #2
    |
note: method defined here
   --> ...\lib.rs:107:8
    |
107 |     fn hash_password(&self, password: &[u8]) -> Result<H> {
    |        ^^^^^^^^^^^^^
help: remove the extra argument
    |
  8 -         .hash_password(b"correct horse battery", &salt)
  8 +         .hash_password(b"correct horse battery")
    |

Some errors have detailed explanations: E0061, E0432.
For more information about an error, try `rustc --explain E0061`.
error: could not compile `rustnote` (example "e02_old_api") due to 2 previous errors
```

- **何が起きたか:** エラーが2つ出ました。1つ目の `E0432` は「`SaltString` と `rand_core` を読み込めません」です。argon2 0.6 では、salt を自分で作る必要がなくなり、`SaltString` は、この場所（`password_hash` の直下）にはありません。`rand_core` も、`note:` のとおり、`rand_core` の機能を有効にしたときだけ使えるようになっています。2つ目の `E0061` は「`hash_password` は引数を1つとるのに、2つ渡されました」です。0.6 の `hash_password` は、パスワードだけを受け取り、salt は中で作ります（`note:` が、`fn hash_password(&self, password: &[u8])` を指しています）。

- **直し方**: 0.6 の書き方は、`Argon2::default().hash_password(b"…")` だけです。RustNote の `auth.rs` と、例2-1の形を参考にします。確かめたら、このファイルは消しておきます。
- **学べること**: 外のクレートの使い方は、版によって変わります。記事やサンプルを写すときは、その記事が使っている版と、自分の `Cargo.toml` の版（第13巻 UNIT 01）が同じかを、まず確かめます。

エラー2（エラーが出ない間違い）：パスワードを、そのまま保存する

もし `users` 表に、パスワードをそのまま（平文で）保存していても、登録もログインも、問題なく動きます。エラーは1つも出ません。困るのは、DB のファイル・バックアップ・ログのどれかが**漏れた日**です。

- 全員のパスワードが、その日のうちに読まれます。多くの人は、同じパスワードをほかのサービスでも使っているので、被害は RustNote の外にも広がります。
- 「暗号化して保存すれば大丈夫」とも考えがちですが、暗号化は鍵で元に戻せる変換です。鍵はサーバーの中のどこかにあるので、サーバーごと盗まれれば、戻されてしまいます。パスワードは、**戻せない**ハッシュにして保存します。
- 例2-3の1つ目のテストは、「`password_hash` にパスワードそのものが入っていない」ことを確かめています。このような**守りのテスト**を残しておくと、誰かがうっかり平文の保存に変えたときに気づけます（第11巻 UNIT 06（準備中））。

RustNote ではここで使う

パスワードをハッシュにする関数と、照合する関数は、`src/auth.rs` にあります（抜き出しています）。

```
/// パスワードを Argon2id でハッシュ化し、PHC 文字列（$argon2id$v=19$...）にする。
/// 塩（salt）は argon2 クレートが毎回ランダムに作る。
pub async fn hash_password(password: &Password) -> Result<String, AppError> {
    let password = password.clone();
    // 重い計算なので、非同期の処理を止めないよう別スレッドで行う
    let joined = tokio::task::spawn_blocking(move || {
        Argon2::default()
            .hash_password(password.as_str().as_bytes())
            .map(|hash| hash.to_string())
    })
    .await;
    match joined {
        Ok(Ok(phc)) => Ok(phc),
        Ok(Err(error)) => Err(AppError::internal(error)),
        Err(error) => Err(AppError::internal(error)),
    }
}
```

- 例2-2で見たとおり、Argon2id は1回に約 26 ミリ秒（debug ビルドでは約 400 ミリ秒）かかる重い計算です。async の処理の中でそのまま計算すると、そのあいだ、同じスレッドのほかのリクエストが止まってしまいます。そこで `spawn_blocking` で、重い計算用の別のスレッドに任せています（第5巻 UNIT 07）。

- `hash_password` は、検査済みの `Password` 型しか受け取りません。8〜128 文字の決まりを通ったパスワードだけが、ハッシュになります（第11巻 UNIT 01 の問3）。

登録していないメールアドレスでログインしようとしたときのために、ダミーのハッシュも用意してあります（抜き出しています）。

```
/// 登録されていないメールでも同じくらい時間をかけるための、ダミーのハッシュ。
/// （すぐ返すと「このメールは未登録だ」と応答時間から推測されてしまう）
static DUMMY_HASH: LazyLock<String> = LazyLock::new(|| {
    Argon2::default()
        .hash_password(b"rustnote-dummy-password")
        .expect("ダミーのハッシュを作れません")
        .to_string()
});
```

- メールアドレスが登録されていないときも、このダミーのハッシュで照合の計算を1回して、時間をそろえます。すぐに「違います」と返すと、応答の速さから「このメールは登録されていない」と分かってしまうからです。問2で、本当に時間がそろっているかを測ります。

練習問題

問1: 例2-1の PHC 文字列の `m=19456,t=2,p=1` は、それぞれ何を表していますか。`m=19456` は、何 MiB のメモリでしょうか。

問2: alice を登録してから、「登録していないメールアドレス」と「alice のメールアドレスで、違うパスワード」で、それぞれ3回ずつログインに失敗し、1回ごとにかかった時間を表示してください。2つの時間は、そろっているでしょうか。

問3: 7 文字のパスワード（短すぎる）で登録しようとしたとき、返ってくる画面に、入力したパスワードが残っているかを確かめてください。メールアドレスはどうでしょうか。

解答と解説

問1の解答:

- `m` は、計算に使うメモリの大きさ（KiB）です。19456 KiB ÷ 1024 = **19 MiB** です。
- `t` は、メモリ全体を何回くり返して計算するか（2 回）、`p` は、計算をいくつ並べて動かすか（1 つ）です。
- この3つは、OWASP が Argon2id の最低の設定として勧めている値と同じで、argon2 0.6 の `Argon2::default()` が使う値です。重くするほど総当たりに強くなりますが、ログインのたびの計算も重くなり、同時にたくさんログインされたときのメモリも増えます（1回 19 MiB なので、同時に 10 人なら 190 MiB）。サーバーの大きさと相談して決めます（第12巻 UNIT 09（準備中））。

問2の解答:

`tests/u02_q2.rs` です。

```rust
//! 問2の解答: 「登録していないメール」と「パスワードが違う」で、ログインの失敗にかかる時間を比べる。
//! 実行: cargo test --test u02_q2 -- --nocapture

mod common;

use std::time::Instant;

use common::test_app;

#[tokio::test]
async fn failed_logins_take_similar_time() {
    let app = test_app().await;
    app.sign_up("alice@example.com", "correct horse battery")
        .await;

    let cases = [
        (
            "登録していないメール",
            "email=nobody%40example.com&password=wrong-password",
        ),
        (
            "パスワードが違う",
            "email=alice%40example.com&password=wrong-password",
        ),
    ];
    for (label, form) in cases {
        let mut times = Vec::new();
        for _ in 0..3 {
            let started = Instant::now();
            let response = app.post_form("/login", form, None).await;
            times.push(format!(
                "{} ({} ms) ",
                response.status().as_u16(),
                started.elapsed().as_millis()
            ));
        }
        println!("{label}: {}", times.join(" / "));
    }
}
```

```
cargo test --test u02_q2 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.65s
     Running tests\u02_q2.rs (target\debug\deps\u02_q2-6804b1bc9d1c3136.exe)

running 1 test
POST /signup -> 303 See Other (415.0974ms)
POST /login -> 401 Unauthorized (804.4866ms)
POST /login -> 401 Unauthorized (380.8674ms)
POST /login -> 401 Unauthorized (395.7334ms)
登録していないメール: 401 (804 ms)  / 401 (381 ms)  / 401 (396 ms)
POST /login -> 401 Unauthorized (379.5492ms)
POST /login -> 401 Unauthorized (406.2936ms)
POST /login -> 401 Unauthorized (398.234ms)
パスワードが違う: 401 (379 ms)  / 401 (406 ms)  / 401 (398 ms)
test failed_logins_take_similar_time ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
3.21s
```

- 「登録していないメール」の2回目・3回目と、「パスワードが違う」の3回は、どれも約 380〜410 ミリ秒（debug ビルド）で、そろっています。ダミーのハッシュのおかげで、時間から「登録されているかどうか」は分かりません。
- ところが、「**登録していないメール」の1回目だけ、804 ミリ秒**と、約2倍かかりました。`DUMMY_HASH` は `LazyLock`（最初に使われたときに一度だけ作る）なので、最初の1回は「ダミーのハッシュを作る計算」と「照合の計算」の2回分の時間がかかるのです。サーバーを起動してから最初の1回だけですが、「登録していないメールの1回目は遅い」という差が残ります。
- これは、RustNote に残っている改善点です。起動するときに、先に一度 `DUMMY_HASH` を作っておけば（`LazyLock::force(&DUMMY_HASH)` を呼ぶなど）、この差は無くなります（RustNote のコードは変えていません）。**守りの仕組みも、測って確かめる**と、こうした抜けが見つかります。

問3の解答:

`tests/u02_q3.rs` です。

```rust
//! 問3の解答: 登録のフォームで断られたとき、入力したパスワードが画面に残るかを確かめる。
//! 実行: cargo test --test u02_q3 -- --nocapture

mod common;

use common::body_text;
use common::test_app;

#[tokio::test]
async fn password_is_not_echoed_back() {
    let app = test_app().await;
    // 7文字のパスワード（短すぎる）で登録しようとする
    let form = "email=alice%40example.com&password=short77&password_confirm=short77";
    let response = app.post_form("/signup", form, None).await;
    let status = response.status();
    let html = body_text(response).await;
    println!("登録の結果: {status}");
    println!(
        "エラーの文: {}",
        html.contains("パスワードは8文字以上にしてください")
    );
    println!(
        "メールアドレスは入力欄に残る: {}",
        html.contains(r#"value="alice@example.com""#)
    );
    println!(
        "パスワードが画面のどこかにある: {}",
        html.contains("short77")
    );
}
```

```
cargo test --test u02_q3 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.81s
     Running tests\u02_q3.rs (target\debug\deps\u02_q3-91faee38663bf341.exe)

running 1 test
POST /signup -> 400 Bad Request (1.9691ms)
登録の結果: 400 Bad Request
エラーの文: true
メールアドレスは入力欄に残る: true
パスワードが画面のどこかにある: false
test password_is_not_echoed_back ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.03s
```

- 400 で、「パスワードは8文字以上にしてください」が出ました。メールアドレスは入力欄（`value="alice@example.com"`）に残りますが、パスワード（`short77`）は、画面のどこにもありません。
- パスワードを入力欄に入れ直して返すと、HTML の中にパスワードがそのまま入ります。HTML は、ブラウザの履歴や、途中の仕組みに残ることがあるので、パスワードは**返さない**のが決まりです。利用者は、パスワードだけ打ち直すことになりますが、安全のための手間です。

まとめ

- パスワードは、暗号化（戻せる）ではなく、ハッシュ（戻せない）にして保存する。RustNote は Argon2id を使い、salt を毎回ランダムに作るので、同じパスワードでも人ごとにハッシュが違う。
- PHC 文字列には、方式・版・重さの設定・salt・ハッシュが入っていて、照合に必要なものはこれだけで足りる。重さの設定（m=19456, t=2, p=1）は、総当たりを何十万倍も遅くする。
- 保存したハッシュが読めないときは 500 にして、原因はログにだけ残す。登録していないメールでもダミーのハッシュで時間をそろえるが、最初の1回だけは遅いという抜けがあることを、測って確かめた。

この巻のまとめ

この版に収録した UNIT 01〜02 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: 認証・セッション・認可と脅威モデル

- ☐ 認証（誰か）・セッション（同じ人か）・認可（してよいか）の違いを、コンサート会場のたとえで説明できる
- ☐ RustNote で、ログインしていないと 303 でログイン画面へ、他人のノートは 404 になることを、テストで確かめられる
- ☐ クッキーに入るトークン（46 文字）と、DB に保存されるハッシュ（32 バイト）の違いと、ハッシュだけを保存する理由を説明できる
- ☐ `__Host-` の名前と、`HttpOnly`・`Secure`・`SameSite=Lax`・`Max-Age` の意味を言える
- ☐ ほかのサイトからの POST が、Topcoat の Origin の検査で 403 になり、何も保存されないことを確かめられる。ヘッダーが無いリクエストが通る理由も言える
- ☐ 持ち主を渡さずにノートを探すと、`E0308`・`E0061` でコンパイルが止まる理由と、SQL に持ち主の条件を書き忘れると IDOR になることを説明できる
- ☐ 守るもの・攻める人・入口・信頼の境目を書き出した脅威モデルの表を作り、守りが無い所（ログインの回数の制限）も書ける

UNIT 02: パスワード・Argon2id・salt

- ☐ パスワードを暗号化ではなくハッシュにして保存する理由と、salt の役目を、ミキサーと隠し味のたとえで説明できる
- ☐ PHC 文字列 `$argon2id$v=19$m=19456,t=2,p=1$…` の5つの部分（方式・版・重さの設定・salt・ハッシュ）を読める
- ☐ 同じパスワードでもハッシュが毎回違うのに、照合は成功する理由を言える

- ☐ 速いハッシュと Argon2id の速さ（1秒に約 870 万回と 39 回）の違いから、わざと重いハッシュを使う理由を説明できる
- ☐ argon2 0.5 の書き方を写したときの `E0432`・`E0061` を読み、版を確かめてから写すことを言える
- ☐ 保存したハッシュが壊れているとログインが 500 になる理由と、ダミーのハッシュで失敗の時間をそろえる仕組み（と、最初の1回だけ遅い抜け）を説明できる

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03（登録・メールのそろえ方・重複）・UNIT 04（ログイン・同じ文で断る・回数の制限）・UNIT 05（Topcoat のセッションの一生）・UNIT 06（クッキー・CSRF・Origin・SameSite）・UNIT 07（認可・IDOR・持ち主の境目）・UNIT 08（Better Auth との比較と、おすすめの構成）・UNIT 09（OIDC・外の認証サービス・MFA・回復）・UNIT 10（守りのテスト・監査・事故の対応）は準備中で、次の更新から順に追加します。それまでの間、RustNote の認証の仕組みは、`Rust/rustnote/src/auth.rs` の先頭のコメントで確かめられます。

用語ミニ辞典

この巻（UNIT 01〜02）に出てきた用語です。

UNIT 01 の用語

用語	意味
認証（authentication）	相手が、名乗っているとおりの人かを確かめること
認可（authorization）	確かめた相手に、その操作をしてよいかを決めること
セッション	一度認証した人を、次のリクエストからも同じ人と分かるようにする仕組み
トークン	セッションを表す、長いランダムな値。RustNote ではクッキー `__Host-session` に入る
`token_hash`	トークンの SHA-256。DB の `sessions` 表には、これだけを保存する
脅威モデル	守るもの・攻める人・入口・信頼の境目を書き出し、ありうる攻撃を整理したもの
信頼の境目	ここより外から来た値は信じずに確かめ直す、という線
`__Host-`	クッキーの名前の頭に付ける決まり。`Secure`・`Path=/`・ドメインの指定なしのときだけ、ブラウザが受け付ける
`HttpOnly` / `Secure` / `SameSite=Lax`	JavaScript から読めない／HTTPS のときだけ送る／ほかのサイトからの POST には付けない、というクッキーの属性
CSRF	ほかのサイトから、ログイン中の人のブラウザに、勝手にリクエストを送らせる攻撃
`Origin` / `Sec-Fetch-Site`	ブラウザが自動で付けるヘッダー。どのサイトのページから送ったかを表す
403 Forbidden	「その要求は受け付けない」という返事。Topcoat は、ほかのサイトからの POST にこれを返す
IDOR	ID を変えるだけで、他人のデータを直接見られてしまう弱さ
E0061	メソッドや関数に渡した引数の数が違うエラー
E0308	型が合わないエラー（ここでは `UserId` の場所に `NoteId` を渡した）

UNIT 02 の用語

用語	意味
ハッシュ	どんな長さのデータからでも、決まった長さの値を作る計算。値から元のデータには戻せない
暗号化	鍵があれば元に戻せる変換。パスワードの保存には使わない
平文（へいぶん）	ハッシュにも暗号化にもしていない、そのままの文字
salt（ソルト）	ハッシュにする前にパスワードに混ぜる、毎回ランダムな値。ハッシュと一緒に保存する
Argon2id	パスワード用の、わざと重く、メモリを多く使うハッシュの方式。RustNote が使う
PHC 文字列	方式・版・重さの設定・salt・ハッシュを `$` で区切って1つにした文字列。`users.password_hash` に入る
`m` / `t` / `p`	Argon2 の重さの設定。メモリ（KiB）・くり返す回数・並べて動かす数
Base64	バイトの並びを、英数字と `+`・`/` で表す書き方。salt とハッシュは、これで文字にしてある
総当たり	考えられるパスワードを、片っ端から試す攻撃
`spawn_blocking`	重い計算を、async の処理とは別のスレッドに任せる tokio の関数
`LazyLock`	最初に使われたときに、一度だけ値を作る入れ物。RustNote の `DUMMY_HASH` に使う
E0432	`use` で読み込もうとしたものが見つからないエラー